Lab 07: RSA Decryption with the Chinese Remainder Theorem

IT4010E — Introduction to Cryptography and Security

SoICT — Hanoi University of Science and Technology

Idea from the course lecturer, written with AI assistance.

Contents

1 Objectives **2**

2 Background **2**

Step 1 — Transformation to the CRT domain 2

Step 2 — Exponentiation in the CRT domain 2

Step 3 — Inverse transformation (recombination) 3

Why is it faster? 3

3 Worked Example **3**

4 Test Data **4**

4.1 Data given in the exercise 4

4.2 Data file 4

5 Lab Tasks **5**

5.1 Task 1 — Standard RSA decryption (15 points) 5

5.2 Task 2 — RSA decryption with CRT (35 points) 5

5.3 Task 3 — Testing (25 points) 6

5.4 Task 4 — Execution time comparison (25 points) 6

6 Deliverables **6**

7 Appendix: Code Skeleton **7**

8 References **8**

1 Objectives

After completing this lab, you should be able to:

1. Implement standard RSA decryption $x = y^d \bmod n$.
2. Implement RSA decryption with the Chinese Remainder Theorem (CRT), following the three steps seen in class.
3. Check that both methods give the same plaintext on small and large keys.
4. Measure how much faster the CRT method is in practice.

This lab is done in groups. Use Python 3.9 or later; Python integers have arbitrary precision, so no external library is needed. The built-in functions `pow(a, b, m)` and `pow(a, -1, m)` are allowed.

2 Background

RSA decryption computes

$$x = y^d \bmod n, \qquad n = pq.$$

When n is large (for example 2048 bits), this computation is expensive, because the private exponent d is as long as n. The owner of the private key also knows p and q. Instead of one large exponentiation modulo n, we can perform two smaller exponentiations modulo p and modulo q, and then recombine the results. The same trick speeds up RSA signature generation.

Decryption with the CRT has three steps.

Step 1 — Transformation to the CRT domain

Split the ciphertext y into two smaller residues:

$$y_p = y \bmod p, \qquad y_q = y \bmod q.$$

Step 2 — Exponentiation in the CRT domain

Reduce the private exponent:

$$d_p = d \bmod (p-1), \qquad d_q = d \bmod (q-1),$$

then compute two modular exponentiations:

$$x_p = y_p^{d_p} \bmod p, \qquad x_q = y_q^{d_q} \bmod q.$$

Each exponentiation works with numbers of about half the size of n.

Step 3 — Inverse transformation (recombination)

Recombine the results:

$$x \equiv [q \cdot c_p]\, x_p + [p \cdot c_q]\, x_q \pmod{n}, \qquad c_p = q^{-1} \bmod p, \quad c_q = p^{-1} \bmod q.$$

The values d_p, d_q, c_p, c_q depend only on the key, so they can be precomputed once and reused for every decryption.

Why is it faster?

Let k be the bit length of n. With schoolbook arithmetic, one exponentiation modulo a k-bit number with a k-bit exponent costs roughly k^3 bit operations. In Step 2, both the modulus and the exponent have about $k/2$ bits, so each exponentiation costs about $(k/2)^3 = k^3/8$. Two of them cost about $k^3/4$, so we expect a speed-up close to 4. Steps 1 and 3 are cheap in comparison.

3 Worked Example

Let $p = 11$, $q = 13$, $n = 143$, $e = 7$ and $d = 103$. We decrypt the ciphertext $y = 15$.

Step	Quantity	Computation	Value
1	y_p	$15 \bmod 11$	4
1	y_q	$15 \bmod 13$	2
2	d_p	$103 \bmod 10$	3
2	d_q	$103 \bmod 12$	7
2	x_p	$4^3 \bmod 11$	9
2	x_q	$2^7 \bmod 13$	11
3	c_p	$13^{-1} \bmod 11$	6
3	c_q	$11^{-1} \bmod 13$	6
3	x	$78 \cdot 9 + 66 \cdot 11 = 1428 \bmod 143$	141

The plaintext is $x = 141$. Indeed $15^{103} \bmod 143 = 141$, and re-encrypting gives $141^7 \bmod 143 = 15$.

4 Test Data

4.1 Data given in the exercise

Use the following key and ciphertext:

$$
\begin{aligned}
p &= 12345678901234567890123456869,\\
q &= 98765432109876543210987654323,\\
n &= p \cdot q,\\
e &= 65537,\\
d &= 183037555140763297287823421841341095154128759392745892977,\\
y &= 12345678901234567890.
\end{aligned}
$$

The modulus n has 190 bits. You can check your answer yourself: if x is correct, then $x^e \bmod n = y$.

4.2 Data file

The file `lab07_testdata.json` contains this key together with larger keys, so that you do not need to generate RSA keys yourself:

Name	Modulus size	e	Number of vectors
`toy-143`	8 bits	7	8
`given-190`	190 bits	65537	1
`rsa-1024`	1024 bits	65537	10
`rsa-2048`	2048 bits	65537	10
`rsa-4096`	4096 bits	65537	10

Each key is a JSON object with the fields `n`, `e`, `d`, `p`, `q`, the expected CRT parameters `crt = {dp, dq, cp, cq}`, and a list `vectors` of pairs `{label, x, y}` where $y = x^e \bmod n$. Some vectors are edge cases ($x = 0$, $x = 1$, $x = n - 1$, and x a multiple of p or of q). The numbers are stored as JSON integers, so `json.load` returns ordinary Python integers:

```python
import json
with open("lab07_testdata.json") as f:
    keys = {k["name"]: k for k in json.load(f)["keys"]}
k = keys["given-190"]
print(k["n"], k["vectors"][0]["y"])
```

5 Lab Tasks

Write your code in `lab07_rsa_crt.py`. A skeleton is given in the Appendix.

5.1 Task 1 — Standard RSA decryption (15 points)

Implement `rsa_decrypt(y, d, n)`, which returns $y^d \bmod n$.

5.2 Task 2 — RSA decryption with CRT (35 points)

1. Implement `crt_precompute(d, p, q)`, which returns the tuple `(dp, dq, cp, cq)`.
2. Implement `rsa_decrypt_crt(y, d, p, q, pre=None)` following Steps 1–3. If `pre` is given, use these precomputed values instead of computing them again.
3. For the key `toy-143` and $y = 15$, print all intermediate values $y_p, y_q, d_p, d_q, x_p, x_q, c_p, c_q$ and the result x. They must match the table of the worked example.

5.3 Task 3 — Testing (25 points)

1. Decrypt the ciphertext y of the exercise (key `given-190`) with both functions. Print x and check that $x^e \bmod n = y$.
2. For every key and every vector in `lab07_testdata.json`, check that `crt_precompute` returns the field `crt` of the key, and that both `rsa_decrypt` and `rsa_decrypt_crt` return `x`. Print one line per key, for example `rsa-2048: 10/10 vectors OK`.

5.4 Task 4 — Execution time comparison (25 points)

1. For the keys `rsa-1024`, `rsa-2048` and `rsa-4096`, decrypt the 10 ciphertexts of the key 10 times (100 decryptions) with `rsa_decrypt`, then with `rsa_decrypt_crt` (with precomputed values). Measure the average time per decryption with `time.perf_counter()`.
2. Report the results in a table of the following form:

Key size	Standard (ms)	CRT (ms)	Speed-up
1024			
2048			
4096			

3. In a few sentences, compare the measured speed-up with the expected value of 4.

6 Deliverables

Each group submits one `.zip` file named `Lab07_Group<Number>.zip` containing:

- Source code: all required functions in well-structured `.py` files. Running `python3 lab07_rsa_crt.py` must run all tests and print the results. The program reads `lab07_testdata.json` from the same folder; do not include this file in your submission.
- `README.md`: a short report that contains
  - the test results with small primes (`toy-143`) and large primes (`given-190` and the 1024–4096-bit keys), including the plaintext x obtained for the exercise ciphertext;
  - the timing table of Task 4 and your comparison between the standard and the CRT method;
  - the list of all group members (full names and student IDs).

You may discuss ideas with other groups, but the code and the report must be your group's own work.

7 Appendix: Code Skeleton

```python
"""IT4010E - Lab 07: RSA Decryption with the CRT"""
import json
import time

DATA_FILE = "lab07_testdata.json"


def load_keys(path: str = DATA_FILE) -> dict:
    with open(path) as f:
        return {k["name"]: k for k in json.load(f)["keys"]}


# ---------- Task 1 ----------
def rsa_decrypt(y: int, d: int, n: int) -> int:
    """Standard RSA decryption: y^d mod n."""
    # TODO
    raise NotImplementedError


# ---------- Task 2 ----------
def crt_precompute(d: int, p: int, q: int) -> tuple[int, int, int, int
    ]:
    """Return (dp, dq, cp, cq)."""
    # TODO
    raise NotImplementedError


def rsa_decrypt_crt(y: int, d: int, p: int, q: int, pre=None) -> int:
    """RSA decryption using the CRT."""
    if pre is None:
        pre = crt_precompute(d, p, q)
    dp, dq, cp, cq = pre
    # Step 1: transformation to the CRT domain
    # Step 2: exponentiation in the CRT domain
    # Step 3: inverse transformation (recombination)
    # TODO
    raise NotImplementedError


# ---------- Task 3 ----------
def test_given(keys: dict) -> None:
    k = keys["given-190"]
    y = k["vectors"][0]["y"]
```

```python
    x1 = rsa_decrypt(y, k["d"], k["n"])
    x2 = rsa_decrypt_crt(y, k["d"], k["p"], k["q"])
    print("x (standard) =", x1)
    print("x (CRT)      =", x2)
    print("check x^e mod n == y:", pow(x2, k["e"], k["n"]) == y)


def test_all(keys: dict) -> None:
    for name, k in keys.items():
        n, d, p, q = k["n"], k["d"], k["p"], k["q"]
        c = k["crt"]
        pre = crt_precompute(d, p, q)
        assert pre == (c["dp"], c["dq"], c["cp"], c["cq"]), name
        ok = 0
        for v in k["vectors"]:
            if (rsa_decrypt(v["y"], d, n) == v["x"]
                    and rsa_decrypt_crt(v["y"], d, p, q, pre) == v["x"]):
                ok += 1
            else:
                print(f"  FAIL {name}: {v['label']}")
        print(f"{name}: {ok}/{len(k['vectors'])} vectors OK")


# ---------- Task 4 ----------
def benchmark(keys: dict, repeat: int = 10) -> None:
    # TODO: average time of rsa_decrypt and rsa_decrypt_crt
    #       on rsa-1024, rsa-2048 and rsa-4096
    pass


if __name__ == "__main__":
    keys = load_keys()
    test_given(keys)
    test_all(keys)
    benchmark(keys)
```

8 References

1. C. Paar, J. Pelzl, *Understanding Cryptography*, Springer, 2010 — Section 7.5.1, “Fast Decryption with the Chinese Remainder Theorem”.
2. K. Moriarty et al., *PKCS #1: RSA Cryptography Specifications Version 2.2*, RFC 8017, 2016 — Section 3.2 and Section 5.1.2.